

РОАДМАП ИНЖЕНЕРА

Как стать AI Engineer в 2026

От нуля до продакшн-агентов
за 17 недель

Agentic AI & Harness Engineering
6 фаз, конкретные проекты,
канонические источники

Перевод и адаптация для @prompt_design

Содержание

Введение	3
Чем занимается Agent Engineer в 2026	4
Бесплатные ресурсы на весь роадмап	5
Инженерные блоги	5
Бесплатные курсы	5
YouTube-каналы и доклады	6
Рассылки	6
Open-source репозитории, которые стоит изучать	6
Подкасты в дорогу	7
Сообщества	7
Фаза 0. Основы (1–2 недели)	8
Что учить	8
1. Augmented LLM и разница workflow vs agent	8
2. Контекст-инжиниринг как дисциплина	8
3. Обязка как операционная система	9
4. Состояние поля в 2026	9
Практический проект	9
Чекпоинт Фазы 0	9
Фаза 1. Первый простой агент (2–3 недели)	11
Что учить	11
1. Цикл агента с нуля	11
2. Claude Agent SDK как эталонная обязка	11
3. Отгрузите что-то маленькое	12
Чекпоинт Фазы 1	12
Фаза 2. Настоящий агент с нормальной архитектурой (3–4 недели)	13
Почему этот стек, а не Pydantic AI, OpenAI Agents SDK или CrewAI	13
Что учить	13
1. LangGraph runtime	13
2. Middleware как слой кастомизации	13
3. Tools, MCP и паттерн code execution	14
4. Память без vector DB	14
Практический проект: «research analyst» deep agent	14
Чекпоинт Фазы 2	15
Фаза 3. Соберите слой обязки сами (3–4 недели)	16
Что учить	16
1. На что разбирается обязка	16
2. Durable execution как add-on	17
Практический проект: мини-обязка ~1500 строк Python	17
Чекпоинт Фазы 3	17
Фаза 4. Слой оценок и регрессионный harness (3–4 недели)	18
Что учить	18
1. Выберите ровно одну платформу observability	18

2. Четыре типа оценок, которые надо реализовать	18
Практический проект: регрессионная обвязка вокруг Phase 2-агента	19
Чекпоинт Фазы 4	19
Фаза 5. Production hardening (постоянно)	21
Что учить	21
1. Дисциплина стоимости	21
2. Latency	21
3. Безопасность и песочницы	21
4. Мониторинг и drift	22
5. Resilience	22
Чекпоинт Фазы 5	22
Рекомендации	24
Если учить только один фреймворк	24
Если изучать одну обвязку как референс	24
Если читать одно по контексту	24
Если выбрать один observability-инструмент	24
Что пропустить в 2026	24
Использовать только под конкретную причину	24
Бенчмарки на закладку (цифры май 2026)	25
Тайминг для технически крепкого инженера, нового в агентах	25
Подводные камни	26
Бенчмарки – движущиеся цели и частично игровые	26
Мульти-агент переоценен для большинства задач	26
Флажки спекуляции в источниках 2026	26
Ландшафт фреймворков может снова сдвинуться	26
МСР в продакшне еще шероховат	26
Поведение моделей меняется между точечными релизами	26
Eval-сюта будет гнить	26
Часть источников – vendor marketing	27
Заключение	28
Что я хочу, чтобы вы вынесли	28

Введение

В выходные на коленке можно собрать следующий миллиардный продукт. Главное – знать, какой навык для этого освоить.

Это **agentic AI и harness engineering** – инженерия обвязки агентов.

Если у вас нет времени читать 7800+ слов, можно отдать ссылку на машиночитаемую версию роадмапа своему агенту и получить персонализированную версию: <https://raw.githubusercontent.com/codejunkie99/agent-roadmap-2026/main/AGENT.md>

Проблема в том, что у большинства инженеров нет ясной картины, что учить.

Кто-то берет CrewAI, потому что в Twitter ролевые демо смотрятся эффектно. Кто-то гонится за каждым новым фреймворком и не доводит до конца ни один. Третьи прыгают сразу в мульти-агентные системы, не разобравшись с контекстом, инструментами, обвязкой и оценкой.

Результат, как правило, один: туризм по фреймворкам и почти ноль продакшн-навыка.

Если ваша цель – стать агентным инженером в 2026 году, не нужно учить 12 фреймворков. Нужно научиться **строить, обвязывать, оценивать и выпускать в продакшн настоящие агентные системы**.

Это значит научиться:

- собирать агентов на полноценном рантайме оркестрации вроде LangGraph
- работать с Claude Agent SDK как с эталонной обвязкой
- грамотно инженерить контекст по схеме Write/Select/Compress/Isolate
- писать инструменты, которые модель выбирает правильно
- добавлять память, durability и песочницы для продакшн-нагрузки
- собирать оценки, проверки траекторий, регрессионные гейты в CI
- выпускать агентов, которые переживают столкновение с реальными пользователями и реальной стоимостью

Этот гайд – 6-фазный роадмап на основе того, что было выпущено в конце 2025-го и начале 2026-го.

Текст на 7000+ слов опирается только на первоисточники. Главная ценность – в том, что у каждой фазы есть конкретный проект, канонический список чтения и нужные ресурсы.

При работе по 17 недель сфокусированно вы выходите на уровень инженера, который может вести продакшн-фичу с агентом от начала до конца.

Чем занимается Agent Engineer в 2026

Многие, услышав «AI agent engineer», представляют человека, который соединяет роли в CrewAI и говорит «отгружено».

В реальности современный agent engineer делает кое-что более практичное. Он строит, обязывает и эксплуатирует агентные системы поверх frontier-моделей. В основном это:

- проектирование цикла агента и диспетчеризации инструментов
- инженерия контекста по схеме Write/Select/Compress/Isolate
- написание инструментов, которые модель выбирает правильно
- оркестрация суб-агентов с изолированными окнами контекста
- добавление скиллов, памяти, durability и песочниц
- настройка оценок, трейсов и CI-гейтов так, чтобы «лучше» стало измеримым

Та же модель + другая обвязка = совершенно другой результат.

Замер от Anthropic: Opus 4.5 показал 78% на CORE внутри Claude Code и 42% в Smolagents. Та же модель. Точка.

Этот разрыв – и есть harness engineering. Об этом весь роадмап.

Четыре примитива контекста, которые должен знать любой строитель агентов:

- **Write** – скретчпады и memory-файлы
- **Select** – поиск релевантного в момент использования
- **Compress** – суммаризация при заполнении 85–95% окна контекста
- **Isolate** – суб-агенты со своими окнами контекста

Мульти-агентная исследовательская система Anthropic обошла одиночного Opus 4 на 90,2% в breadth-first research именно по этой схеме – ценой ~15-кратного расхода токенов.

На практике в 2026 году имеет смысл глубоко изучить только два стека: **LangGraph 1.0 + Deep Agents** и **Claude Agent SDK**.

Все остальное либо угасает, либо поглощается, либо в продакшне работает хуже этих двух.

Бесплатные ресурсы на весь роадмап

Это блоги, курсы, каналы и рассылки, которые регулярно дают сигнал, а не шум. Подпишитесь на них в Фазе 0, чтобы остальной роадмап ложился на постоянный поток свежих постов, кейсов и обновлений из первоисточников.

Все бесплатно, и обновляется быстрее, чем любой учебник.

Инженерные блоги

- **Anthropic engineering blog** (<https://anthropic.com/engineering>) – если читать только один блог, читайте этот. Контекст-инжиниринг, дизайн обвязок, мульти-агентные исследования, продвинутый tool use, оценки. Все первоисточники, все цитируется по всему роадмапу.
- **LangChain blog** – здесь публично формализуют дисциплину обвязки, middleware и Deep Agents. Читайте все за авторством Lance Martin, Vivek Trivedy и Harrison Chase.
- **OpenAI Cookbook** (GitHub) – рабочие ноутбуки на каждую фичу API. Tool use, structured outputs, оценки, агенты. Печатайте за ним.
- **Hamel Husain** – эссе «Your AI Product Needs Evals», на которое ссылаются все. Все остальное на сайте такого же уровня. Если вы делаете оценки, перечитывайте дважды.
- **Eugene Yan** – «Patterns for Building LLM-based Systems & Products», на который ссылаются все практики. Откалибровано на реальном опыте отгрузок.
- **Lilian Weng** – большие разборы про агентов, prompt engineering, галлюцинации, alignment. Самый ясный синтетический материал в поле.
- **Simon Willison** – ежедневные заметки старшего инженера, который реально отгружает. Хорошо лечит хайп и ловит странные кейсы первыми.
- **Chip Huyen** – ML-системы с первых принципов. «Building LLM applications for production» обязательна перед Фазой 5.
- **Phil Schmid** – практические end-to-end гайды по HuggingFace, Gemini, fine-tuning, деплою. Всегда с кодом.
- **Cameron Wolfe (Deep Learning Focus)** – большие разборы статей. За одно чтение можно догнать целое исследовательское направление.

Бесплатные курсы

- **DeepLearning.AI Short Courses** – короткие курсы 1–2 часа. В Фазе 0 пройдите курс по LangGraph (с LangChain) и «Agentic AI» Эндрю Ына (Reflection, Tool Use, Planning, Multi-Agent design patterns).
- **LangChain Academy: Introduction to LangGraph** – официальный бесплатный курс. State, память, human-in-the-loop, мульти-агенты. Делайте в Фазе 2.
- **Anthropic Interactive Prompt Engineering Tutorial** (GitHub) – девять глав в виде Jupyter-ноутбуков на Claude API. Самый быстрый способ накачать промптинговую мышцу.
- **HuggingFace Agents Course** – покрывает агентов, smolagents, MCP и оценку. Бесплатный сертификат.
- **HuggingFace LLM Course** – основы: токенизация, трансформеры, fine-tuning. Полезный фон, даже если вы строите только на API.
- **MCP Fundamentals on FreeAcademy** – собирать MCP-серверы, подключать к Claude, писать кастомные инструменты. Самый быстрый путь к MCP-грамотности.

YouTube-каналы и доклады

- **Andrej Karpathy** – «Neural Networks: Zero to Hero», там GPT собирается с нуля на чистом Python. Доклад «Vibe Coding to Agentic Engineering» с Sequoia AI Ascent 2026 – самое ясное объяснение, почему harness engineering важен именно сейчас.
- **AI Engineer** – доклады с AI Engineer Summit и World's Fair. Ищите Hamel Husain, swyx, инженеров Anthropic, Erik Schluntz.
- **LangChain** – еженедельные tutorиалы по LangGraph, Deep Agents, middleware и интеграциям. Часто новые фишки появляются здесь раньше всего.
- **Anthropic** – доклады инженеров Anthropic. Разборы мульти-агентных исследований, внутренности Claude Code, Skills.
- **Yannic Kilcher** – разборы статей. Экономит чтение всех arXiv-препринтов.
- **Lex Fridman Podcast (на YouTube)** – длинные интервью с теми, кто строит и исследует AI. Karpathy, Schulman, Sutskever, Amodei.

Рассылки

- **Latent Space (swyx и Alessio)** – технический ньюслеттер для AI-инженеров. Ежедневный AINews, подкаст и ежегодный «AI Engineering Reading List». Если подписаться только на один – этот.
- **The Batch (Andrew Ng)** – еженедельный широкий обзор. Полезно, чтобы заметить, когда что-то новое выходит на радар.
- **Import AI (Jack Clark, со-основатель Anthropic)** – политика плюс research roundup. Ближайшее к стратегическому брифингу по полю.
- **Ben's Bites** – ежедневные новости за пять минут. Только просматривать. Удобно ловить анонсы, которые иначе пропустите.
- **TLDR AI** – ежедневный дайджест, мало шума. Сочетайте с одной из глубоких рассылок выше.
- **AI Engineer Pack (swyx)** – курируемые бесплатные кредиты, инструменты и ресурсы для AI-инженеров. Постоянно обновляется.

Open-source репозитории, которые стоит изучать

- **Anthropic Cookbook (GitHub)** – референс-имплементации каждого workflow-паттерна. Перечитывайте после каждой фазы.
- **OpenAI Cookbook (GitHub)** – то же со стороны OpenAI. Tool use, structured outputs, оценки, агенты.
- **deepagents (LangChain, GitHub)** – эталонная open-source обвязка поверх LangGraph. Читайте middleware-файлы, когда будете писать свою обвязку в Фазе 3.
- **LangGraph examples (GitHub)** – работающие паттерны LangGraph. Supervisor, hierarchical teams, planning, customer-support agent.
- **inspect_evals (GitHub)** – 200+ стандартных оценок как Python-пакет. GAIA, SWE-bench, Cybench, BFCL.
- **awesome-agentic-engineering-resources (GitHub)** – комьюнити-индекс по теме. Закрывает пробелы, которых нет в этом роадмапе.

Подкасты в дорогу

- **Latent Space** – длинные интервью с людьми, которые отгружают поле. Anthropic, OpenAI, LangChain, Modal, E2B.
- **Dwarkesh Podcast** – большие интервью про стратегию AI, способности и политику. Длинные форматы, первоисточники.
- **The TWIML AI Podcast (Sam Charrington)** – еженедельные технические интервью с исследователями и инженерами.
- **Practical AI** – инженерный фокус. Меньше хайпа, больше отгрузок.
- **The MAD Podcast (Matt Turck)** – взгляд фаундера и инвестора на data/AI экосистему. Полезно отслеживать, кто отгружает, а кто только привлекает.

Сообщества

- **LangChain Discord** – ядро команды LangGraph и Deep Agents. Активные #help-каналы.
- **HuggingFace Discord** – крупнейшее open-weights и ML-комьюнити.
- **r/LocalLLaMA** – новости open-weights моделей, бенчмарки, тулинг. Часто быстрее официальных каналов.
- **AI Engineer World's Fair** – профессиональная сеть поля. Вакансии, наем, рабочие группы.
- **Anthropic Discord** – комьюнити разработчиков на Claude. Skills, hooks, MCP-серверы.

На что ориентироваться: в Фазе 0 выберите по одному блогу, рассылке, подкасту и сообществу. Не пытайтесь подписаться сразу на все 40+ источников. Добавляйте новые, только когда старые перестают вас удивлять. Список здесь дан для выбора, а не для галочки.

Фаза 0. Основы (1–2 недели)

Цель фазы: собрать правильные ментальные модели. Не написать пока ни строчки агентного кода, кроме одноразовых скриптов.

Большинство новичков пропускают эту фазу, ныряют в tutorиалы фреймворков и в итоге получают код, который не могут разобрать, когда он падает. Не пропускайте.

Что учить

1. Augmented LLM и разница workflow vs agent

Прежде чем трогать любой фреймворк, разберитесь в пяти workflow-паттернах от Anthropic (prompt chaining, routing, parallelization, orchestrator-worker, evaluator-optimizer) и поймите, почему workflow – это не то же самое, что агент.

- У workflow поток управления зафиксирован вами.
- Агент сам принимает решения о потоке управления внутри цикла.

Это различие спасет вас от того, чтобы городить агентов там, где должны были быть chains.

Ресурсы:

- *Building Effective Agents* – Anthropic (Erik Schluntz, Barry Zhang), декабрь 2024. Пять workflow-паттернов и augmented-LLM. Ссылаются все. Читайте первым.
- *Anthropic Cookbook* (папка `patterns/agents`) – референс-имплементации каждого паттерна как ноутбуки. Печатайте за ними.
- *Simon Willison* – его аннотации к *Building Effective Agents*. Здравый взгляд старшего инженера на ту же статью.

На что фокусироваться: разница между workflow и агентом, ментальная модель augmented LLM, паттерн orchestrator-worker, почему параллелизация почти всегда лучше последовательного рассуждения, и какие режимы сбоя Anthropic явно предупреждает.

2. Контекст-инжиниринг как дисциплина

Prompt engineering как самостоятельный навык в 2026 умер. Замена – **context engineering**: решение, какие токены стоят перед моделью на каждом шаге цикла.

Ресурсы:

- *Effective context engineering for AI agents* – Anthropic, 29 сентября 2025. Прочитайте дважды. Запомните рамку.
- *Context Engineering for Agents* – Lance Martin (LangChain). Та самая Write/Select/Compress/Isolate.
- *How we built our multi-agent research system* – Anthropic, июнь 2025. Эталонная архитектура orchestrator-worker, прирост 90,2% на breadth-first research и оговорка про 15× токенов.
- *Simon Willison* – аннотации к посту про мульти-агентную систему. Здравый взгляд на архитектуру и trade-off по стоимости.

На что фокусироваться: что такое каждый из Write/Select/Compress/Isolate в коде, почему суб-агенты – это примитив изоляции (не параллелизма), и когда применять компакцию vs offload vs суммаризацию.

3. Обязка как операционная система

Ресурсы:

- *The Complete Guide to Harness Engineering* (ClaudeCodeLab) – три уровня эскалации обязки с работающим кодом.
- *Inside the Claude Agents SDK* (ML6) – аналогия CPU/RAM/OS/App плюс цифра 78% vs 42% Opus 4.5, мотивирующая весь роадмап.
- *Building agents with the Claude Agent SDK* (Anthropic) – зачем существует SDK, почему его переименовали из Claude Code SDK.
- *Effective harnesses for long-running agents* (Anthropic), 26 ноября 2025. Праймер по обязкам от самой Anthropic.
- *Harness design for long-running application development* (Anthropic), 24 марта 2026. Продолжение – что меняется, когда сессии растягиваются на часы и дни.
- *How to think about agent frameworks* (Harrison Chase, LangChain) – разница между orchestration framework и абстракцией. Обязательно перед выбором стека.

На что фокусироваться: loop, диспетчеризация инструментов, курирование контекста, persistence, hooks, оркестрация суб-агентов, observability – и как каждое реализовано в любой обязке, с которой столкнетесь.

4. Состояние поля в 2026

Ресурсы:

- *State of Agent Engineering* (LangChain) – 1340 респондентов, ноябрь–декабрь 2025. Запомните цифры: 57% команд в проде, 89% имеют observability, 52% имеют оценки, главный барьер – качество (32%).
- *How to Build an Agent* (LangChain) – модель «smart intern» для скоупинга того, что должен и не должен делать агент.
- *Continual learning for AI agents* (Harrison Chase, LangChain) – три слоя, где агенты реально учатся: веса, промпты, память.

На что фокусироваться: где команды страдают в продакшне (качество, стоимость, надежность), как выглядит средний стек, и где маржинальный час работы окупается лучше всего.

Практический проект

Напишите от руки 2-страничный личный документ, в котором своими словами объясните: workflow vs agent, augmented LLM, четыре примитива контекст-инжиниринга, паттерн orchestrator-worker, разницу harness/model/framework и три ожидаемых режима сбоя в собственном коде.

Этот документ и есть результат фазы. Если не получается написать без подсматривания, значит, вы читали невнимательно.

Чекпоинт Фазы 0

К концу фазы вы должны уметь:

- Объяснить, что такое агент и чем он отличается от workflow, без жаргона фреймворков.

- Назвать четыре примитива контекст-инжиниринга и привести пример каждого на уровне кода.
- Объяснить, почему обвязка в 2026 году вкладывает больше, чем модель.
- Описать паттерн orchestrator-worker и trade-off 15× токенов.
- Выбирать фреймворк по архитектуре, а не по вайбам.

Фаза 1. Первый простой агент (2–3 недели)

Цель фазы: написать tool-using агента дважды. Один раз на сыром Anthropic SDK, второй раз на обвязке Claude Agent SDK. Ощутить разницу между «свой цикл» и «стою на нормальной обвязке».

Это самый дешевый способ понять, что обвязка дает на самом деле.

Что учить

1. Цикл агента с нуля

Цикл – не магия. Вы вызываете модель с сообщениями и инструментами, парсите блоки `tool_use`, исполняете инструменты, прицепляете `tool_result`, повторяете, пока `stop_reason == end_turn`.

Один раз написав это в ~100 строк, любой фреймворк станет читаемым.

Ресурсы:

- *Tutorial: Build a tool-using agent* (Anthropic docs) – референс по `tool_use`, `tool_result`, параллельным вызовам и циклу ответа.
- *Writing tools for agents* (Anthropic) – читать перед дизайном любого инструмента. Описания инструментов и параметров – это инструкция к LLM.
- *Equipping agents for the real world with Agent Skills* (Anthropic) – паттерн progressive disclosure, объясненный командой, написавшей спецификацию.

На что фокусироваться: как завершается цикл запрос/ответ, что значат разные `stop_reason`, как кодируются параллельные вызовы, восстановление при ошибках инструмента, как написать описание инструмента, чтобы модель его выбирала правильно.

Практика: соберите «from scratch» агента в 100 строк через `anthropic.messages.create` с tool spec. Три инструмента: `web_search` через Tavily или Firecrawl, `read_file`, `write_file`. Никакого фреймворка. Прогоните на исследовательской задаче и прочитайте каждый шаг трейса.

2. Claude Agent SDK как эталонная обвязка

Claude Agent SDK – та же обвязка, что под капотом Claude Code. Изучайте как референс и используйте как инструмент с первого дня.

Ресурсы:

- *Claude Agent SDK docs* – Python и TypeScript SDK, hooks, суб-агенты, skills и Task-инструмент.
- *Claude Agent SDK, Skills reference* – как работают `SKILL.md`, frontmatter с метаданными, прогрессивная загрузка.
- *claude-code-best-practices* (Muhammad Usman GM, GitHub) – листайте, не копируйте. Полезно посмотреть, что делают реальные пользователи.
- *claude-code-best-practice* (Shan Raishan, GitHub) – компендиум-компаньон с другой подборкой.
- *Evaluating Skills* (LangChain) – как LangChain меряет, реально ли Skill тащит. Полезно после того, как напишете свой первый Skill.

На что фокусироваться: паттерн `CLAUDE.md` как системного промпта, прогрессивная загрузка Skills, hooks `PreToolUse` и `PostToolUse`, спавн суб-агентов через Task-инструмент, как SDK обрабатывает permission-промпты.

Практика: перепишите того же агента на `claude-agent-sdk`. Добавьте `CLAUDE.md` с конвенциями проекта. Добавьте один Skill (папка с `SKILL.md`), описывающий формат «research-summary». Добавьте один `PostToolUse`-хук, автоформатирующий любой файл, который пишет агент. Заспавните одного суб-агента под подзадачу через Task.

3. Отгрузите что-то маленькое

Тьюториалы не считаются. Нужен инструмент, который запускается по расписанию, и чей выход вы реально читаете.

Практический проект: агент с утренним брифингом, который читает локальные Markdown-заметки и пару RSS-фидов, делает суммаризированный брифинг с цитатами и пишет на диск. Закроните через `launchd` или `systemd`. Прогоните неделю. Смотрите, как падает. Чините.

Чекпоинт Фазы 1

К концу фазы вы должны уметь:

- Написать tool-using цикл агента в 100 строк без фреймворка.
- Объяснить, что значат `stop_reason` и как работают параллельные вызовы.
- Собрать того же агента на Claude Agent SDK со Skill, hook и суб-агентом.
- Сформулировать в 200 словах, что обвязка дала бесплатно, что вы писали руками.

Фаза 2. Настоящий агент с нормальной архитектурой (3–4 недели)

Цель фазы: собрать многошагового, persistent, stateful агента на **LangGraph 1.0 + LangChain create_agent + Deep Agents**.

Это стек, который вы скорее всего будете гонять в продакшне. Концептуальная модель (state machine из узлов и ребер, middleware, checkpoint) переносится куда угодно.

Почему этот стек, а не Pydantic AI, OpenAI Agents SDK или CrewAI

- LangGraph – единственный фреймворк в рейтингах Alice Labs и Channel.tel «что отгружают», который сочетает durable execution, checkpointing, human-in-the-loop, first-class observability через LangSmith и middleware.
- `create_agent` (LangChain 1.0, октябрь 2025) – дефолтная фабрика агентов поверх LangGraph runtime. `create_react_agent` устарел.
- Deep Agents (LangChain, август 2025; v0.5 alpha апрель 2026) – batteries-included обвязка поверх. Planning, виртуальная файловая система, суб-агенты, суммаризация, skills. Это ближайший open-source аналог обвязки Claude Code, при этом model-agnostic.

Что учить

1. LangGraph runtime

State-граф из узлов и ребер плюс checkpoint, дающий resume, rewind и fork.

Ресурсы:

- *LangGraph docs* – референс рантайма. Начинайте с concepts, потом quickstart.
- *Doubling down on Deep Agents* (LangChain) – чисто разделяет harness vs framework vs runtime.
- *Context Management for Deep Agents* (LangChain) – паттерн offload tool-ответов на 20K токенов и триггер компакций на 85% окна контекста.
- *On Agent Frameworks and Agent Observability* (LangChain) – почему LangSmith дружит с OTEL и работает без LangChain.
- *Deep Agents v0.5* (LangChain) – релиз-ноуты апреля 2026: асинхронные суб-агенты, мульти-модальная виртуальная ФС, async TODOs.

На что фокусироваться: state-схемы, узлы, ребра, conditional edges, `PostgresSaver`, time-travel debugging, human-in-the-loop interrupts, как компонируется middleware.

2. Middleware как слой кастомизации

Middleware – то, как вы кастомизируете запакованного агента, не форкая его.

Ресурсы:

- *How Middleware Lets You Customize Your Agent Harness* (LangChain), 26 марта 2026. Хуки `before_agent`, `wrap_model_call`, `before_tools`, `after_tools`. Обязательно.
- *Introducing ambient agents* (LangChain) – UX-паттерны фоновых агентов: notify, question, review.

На что фокусироваться: где каждый хук срабатывает в жизненном цикле агента, как `SummarizationMiddleware` и `FilesystemMiddleware` композируются, как написать свой `middleware` в 30 строк, и когда `middleware` – правильный ответ, а когда лучше написать новый узел.

3. Tools, MCP и паттерн code execution

Наивный «загрузить все MCP-инструменты в контекст» сломан. Правильный паттерн – `code execution` с MCP.

Ресурсы:

- *Code execution with MCP* (Anthropic), ноябрь 2025. Сокращение 150K → 2K токенов. Читать перед подключением любого MCP-сервера.
- *Introducing advanced tool use* (Anthropic). `defer_loading: true` срезает 85% токенов на инструменты и поднимает Opus 4.5 на MCP-оценке с 79,5% до 88,1%.
- *Scaling Managed Agents* (Anthropic) – разделение `session`, `harness` и `sandbox`. Читать, даже если не используете `Managed Agents`.
- *Composio docs* (free tier) – 200+ SaaS-интеграций, MCP-шлюз, посредник по `credentials` – они никогда не попадают в контекст модели.
- *Arcade docs* (free tier) – когда нужна тонкая `per-user identity`, а не сервисная авторизация.

На что фокусироваться: `defer_loading`, `code execution` как поверхность инструментов, почему `round-trip JSON` через модель дорог, как `Composio` или `Arcade` брокерят SaaS-авторизацию без утечек `creds` в контекст.

4. Память без vector DB

Ресурсы:

- *Letta MemFS benchmark on LoCoMo* – результат апреля 2026: `filesystem`-память на GPT-4o-mini – 74% на LoCoMo, обогнала специализированные `memory`-инструменты.
- *Mem0 docs* – `user-scoped knowledge memory`. Берите для `cross-session` фактов о пользователе.

На что фокусироваться: три слоя памяти (`thread-scoped` через `PostgresSaver`, `user-scoped` через `Mem0/Zep`, `self-managed` через `Letta`), почему файловая система – правильный дефолт, и почему нельзя тащить `vector DB` до того, как замеры реальную проблему `recall`.

Практический проект: «research analyst» deep agent

- На вход: исследовательский вопрос.
- Lead-агент планирует, пишет `TODO` в виртуальную ФС, спавнит 3 поисковых суб-агента параллельно с изолированным контекстом.
- Суб-агенты вызывают `Tavily` или `Firecrawl`, пишут результаты в файлы, возвращают короткие саммары. Никаких сырых результатов поиска в контекст родителя.
- Citation-агент проверяет утверждения по найденным источникам.
- Writer-агент собирает финальный `Markdown`-отчет с инлайн-цитатами.
- Все состояние `persistится` через `PostgresSaver`. Убейте процесс на середине – продолжите с того же места.

- HITL-interrupt: агент должен спросить подтверждение перед превышением \$1 на токены.
- Все обернуто в один `make demo`, прогоняющий пайплайн от начала до конца.
- README объясняет: какой middleware и почему, у каких суб-агентов изолированный контекст, какая стратегия компакций и какая история durability при kill.
- Приложите URL трейса в LangSmith для одного полного прогона.

Чекпоинт Фазы 2

К концу фазы вы должны уметь:

- Собрать многошагового LangGraph-агента с `PostgresSaver` и HITL-interrupts.
- Использовать middleware Deep Agents (planning, filesystem, sub-agents, summarization) как готовую обвязку.
- Спавнить суб-агентов с изолированным контекстом и возвращать сжатые саммари в родителя.
- Сформулировать стратегию компакций контекста и историю durability при kill.
- Выдать LangSmith-URL с полной мульти-шаговой траекторией.

Фаза 3. Соберите слой обвязки сами (3–4 недели)

Цель фазы: перестать пользоваться готовой обвязкой и собрать свою тонкую. Вы никогда не примете правильные harness-trade-offs в продакшне, пока не соберете свою хотя бы раз.

Это самая высокорычажная фаза в роадмапе.

Что учить

1. На что разбирается обвязка

Сводя middleware Deep Agents, архитектуру Claude Agent SDK и пост Vivek Trivedy про harness engineering, обвязка – это объединение:

- **Loop control.** Цикл model → tools → model.
- **Tool dispatch.** Реестр, валидация схем, параллельные вызовы, восстановление, retries.
- **Context management.** Сборка системного промпта, компакция истории при 85–95% окна, offload tool-ответов от ~20K токенов, prompt caching.
- **Persistence.** Чекпоинт состояния на каждом узле, чтобы можно было resume/rewind/fork.
- **Sub-agent orchestration.** Спавн isolated-context детей, сжатые саммари обратно к родителю.
- **Skills и progressive disclosure.** Подгружать возможности, только когда они нужны.
- **Hooks.** PreToolUse, PostToolUse, PreCompact, Stop, SessionStart (канон от Claude Code).
- **Observability.** OTEL-spans на каждый model call, tool call, sub-agent invocation, с подсчетом токенов и latency.
- **Sandboxing.** Code execution и MCP-вызовы происходят в контейнере, до credentials которого модель не дотягивается.
- **Auth и брокинг секретов.** Учетки никогда не попадают в контекст (паттерн Anthropic Managed Agents).

Ресурсы:

- *The Anatomy of an Agent Harness* (LangChain) – самое чистое разложение компонентов в публичной литературе. Опорный текст всей фазы.
- *Improving Deep Agents with harness engineering* (Vivek Trivedy, LangChain), 17 февраля 2026. Подъем с 30 на 5 место Terminal-Bench 2.0 только за счет смены обвязки, при фиксированной модели GPT-5.2-codex.
- *Better Harness: A Recipe for Harness Hill-Climbing with Evals* (Vivek Trivedy), 29 апреля 2026. Прямое продолжение – self-verification и трейсы как рецепт автономного улучшения обвязки.
- *Inside the Claude Agents SDK* (ML6) – аналогия CPU/RAM/OS/App и сравнение 78% vs 42%.
- *everything-claude-code* (победитель Cerebral Valley × Anthropic, GitHub) – как пример того, где остановиться с фичами.
- *deepagents source* (GitHub) – читайте параллельно своей обвязке как референс. Middleware-файлы – ядро паттерна.

На что фокусироваться: какие компоненты писать самому, какие подтащить, в каком порядке (loop и tool dispatch до суб-агентов до durability до observability).

2. Durable execution как add-on

Ресурсы:

- *Inngest docs* – durable steps и checkpointing GA с декабря 2025. Самый простой путь к durability для Python-обвязки.
- *Temporal Python SDK* – интеграция OpenAI Agents SDK + Temporal вышла в марте 2026. Каждый tool call – durable step.

На что фокусироваться: ключи идемпотентности на шаг, политики retry, что происходит с in-flight вызовами при kill, где должна быть граница чекпоинта (на узел, не на токен).

Практический проект: мини-обвязка ~1500 строк Python

- Loop вокруг `anthropic.messages.create` или `LiteLLM` (для model-agnosticism).
- Реестр инструментов через декоратор Python (`@tool`) с автогенерацией JSON-schema.
- Загрузчик системного промпта в стиле `CLAUDE.md`, читающий `./harness/rules/*.md` с path-glob matching.
- Загрузчик `SKILL.md` с прогрессивной загрузкой (целевые менее 50 токенов метаданных на skill в контексте).
- Прimitives спавна суб-агента с изолированным контекстом, возвращающий summary-строку родителю.
- Filesystem offload: любой результат инструмента >20K токенов пишется в `./workspace/<id>.txt`, в контексте остается путь и preview из 10 строк.
- Авто-компрессия при 85% окна: суммаризация сообщений старше последних 10 ходов.
- Подключаемая система хуков (`pre_tool`, `post_tool`, `stop`).
- OpenTelemetry-трейсинг через `opentelemetry-sdk` в LangSmith или Phoenix (оба понимают OTEL).
- Durable resume: persist истории сообщений и состояния в SQLite после каждого шага, перезагрузка по run ID.
- Опционально: оберните все в Inngest или Temporal так, чтобы каждый tool call стал durable step.

Чекпоинт Фазы 3

К концу фазы вы должны уметь:

- Перечислить десять компонентов современной обвязки и сказать, когда каждый окупается.
- Написать 1500-строчную Python-обвязку с loop, tool dispatch, компрессией контекста, суб-агентами, хуками и OTEL-трейсами.
- Подключить durable execution через Inngest или Temporal так, чтобы kill процесса был обратимым.
- Написать 1000-словный post-mortem с сравнением своей мини-обвязки с Claude Agent SDK и Deep Agents. Что взяли, что вырезали, что бы переделали.

Этот post-mortem – настоящий результат. Код – лишь доказательство.

Фаза 4. Слой оценок и регрессионный harness (3–4 недели)

Цель фазы: сделать вашего агента измеримым. Без этого любое «улучшение» – вайбы.

Здесь застревает большинство инженеров. Они умеют сделать классного агента, но не могут сказать, лучше ли стало после следующего изменения.

Что учить

1. Выберите ровно одну платформу observability

Не запускайте две. Реальные варианты:

- **LangSmith.** Если живете в LangGraph или LangChain. Нативный трейсинг. В марте 2026 добавились Sandboxes, Polly debugging assistant, Skills и Fleet (agent identity/sharing).
- **Braintrust.** Если хотите framework-agnostic CI quality gates, блокирующие PR. Серия B \$80M в феврале 2026. Плоские \$249/мес за неограниченных пользователей против \$39/seat у LangSmith.
- **Arize Phoenix (open source) и Arize AX (managed).** Если хотите OpenTelemetry-native, drift detection и чистую миграцию из OSS в managed.
- **W&B Weave.** Если уже на Weights & Biases в ML. Полные agent trace views, MCP auto-logging, готовится A2A-трейсинг.
- **Inspect (UK AISI).** Для benchmark-grade оценок. GAIA, SWE-bench, Cybench, BFCL поставляются как inspect_evals. Используется внутри Anthropic, DeepMind, Grok.

Ресурсы:

- *LangSmith docs* – production tracing, online evals, эксперименты, новый Polly debugging assistant.
- *Inspect AI annotated notes* (Hamel Husain) – практический рассказ Hamel. Перед установкой Inspect.
- *Inspect docs* – референс фреймворка.
- *inspect_evals* (GitHub) – 200+ оценок как Python-пакет.
- *Braintrust docs* (free tier) – framework-agnostic эксперименты, CI gates, золотые датасеты.
- *Agent Evaluation Readiness Checklist* (LangChain) – 17-минутный чек-лист: error analysis, конструкция датасета, дизайн grader, offline и online оценки, готовность к продакшну. Распечатайте и приклейте к монитору.
- *Quantifying infrastructure noise in agentic coding evals* (Anthropic), 5 февраля 2026. Только flaky sandboxes и сетевой джиттер могут двигать счета на несколько пунктов. Перед тем как доверять любому agent-бенчмарку, читайте.

На что фокусироваться: стратегия trace sampling, online vs offline оценки, разница между метрикой и guardrail, почему CI-рейтинг – паттерн, который превращает оценки из обоев на дашборде в инструмент разработки.

2. Четыре типа оценок, которые надо реализовать

По Anthropic's *Demystifying evals for AI agents*:

- **Single-turn evals.** Дан вход – верный ли выход? Самые дешевые, по возможности детерминированные грейдеры, гонять постоянно.

- **Trajectory evals.** Вызвал ли агент правильную последовательность инструментов с правильными аргументами? Тестируйте single-step, full-turn и multi-turn.
- **LLM-as-judge.** Для open-ended выходов (исследовательские отчеты, ревью кода). Калибруйте против размеченных людьми примеров еженедельно. Anthropic в research-агенте использовали 0,0–1,0 по факт-точности, цитированию, полноте, качеству источников и эффективности инструментов.
- **End-state evals.** Для stateful агентов (DB writes, file edits). Сравнивайте конечное состояние среды с ground truth. Подход tau-bench.

Ресурсы:

- *Demystifying evals for AI agents* (Anthropic) – лучший праймер.
- *Evaluating Deep Agents: Our Learnings* (LangChain) – паттерны single-step, full-turn, multi-turn trajectory evals.
- *How we build evals for Deep Agents* (LangChain) – как реально собирают данные, проектируют метрики, гоняют scoped-evals.
- *Eval awareness in Claude Opus 4.6's BrowseComp performance* (Anthropic), 6 марта 2026. Модели могут понимать, что их оценивают, и вести себя иначе. Читать перед дизайном eval-сюта.
- *Designing AI-resistant technical evaluations* (Anthropic), 21 января 2026. Как проектировать оценки, которые не геймятся теми же моделями, которыми вы их оцениваете.
- *tau²-bench repository* (GitHub) – multi-turn customer-service оценки с policy compliance.
- *Establishing Best Practices for Building Rigorous Agentic Benchmarks* (arXiv) – читать перед своими бенчмарками. SWE-bench, KernelBench и WebArena переоценивают на 5–33%.

На что фокусироваться: как написать детерминированный grader там, где можно; как калибровать LLM-judge против человека; когда `passk` важнее `pass@1`; как обнаруживать и выкидывать загрязненные бенчмарки.

Практический проект: регрессионная обвязка вокруг Phase 2-агента

- Соберите golden-датасет на 30–50 размеченных вручную исследовательских вопросов трех уровней сложности (Level 1/2/3 в стиле GAIA).
- Реализуйте детерминированные грейдеры там, где можно (exact-match для фактических запросов), и LLM-as-judge с 5-критериальным rubric для open-ended.
- Добавьте trajectory eval: спланировал ли агент, заспавнил ли ≥ 2 суб-агента, процитировал ли источники, уложился ли в бюджет?
- Вшейте в GitHub Actions: каждый PR прогоняет полную сьюту. Блок merge, если pass rate на golden-сете падает на ≥ 3 пункта или любой `pass4` снижается.
- Добавьте production sampling: 1% живых трейсов получают auto-grade LLM-as-judge ночью. Алерт на drift.
- Прогоните агента против опубликованного бенчмарка через Inspect: GAIA Level 1 или tau²-bench retail. Сравните цифры с публичными лидербордами.
- Соберите `make eval`, выдающий три артефакта: CI pass/fail summary, LangSmith experiment URL и Inspect log с одной канонической оценкой бенчмарка.

Чекпоинт Фазы 4

К концу фазы вы должны уметь:

- Выбрать одну observability-платформу и защитить выбор архитектурно.
- Реализовать все четыре типа оценок: single-turn, trajectory, LLM-as-judge, end-state.
- Поддерживать golden-датасет, который растет из продакшн-падений, не из синтетики.
- Блокировать PR в CI при регрессии оценок.
- Выдавать `make eval` с CI summary, LangSmith URL и Inspect log.

Главный артефакт – задокументированные режимы сбоя вашего агента. Этот документ и есть продукт.

Фаза 5. Production hardening (постоянно)

Цель фазы: взять все, что собрали, и сделать так, чтобы оно пережило реальных пользователей, реальную стоимость и реальные сбои.

Эта фаза не заканчивается.

Что учить

1. Дисциплина стоимости

- Используйте prompt caching агрессивно. Caching от Anthropic экономит до 90% на повторяющихся префиксах. Кешируйте `CLAUDE.md`, системный промпт и определения инструментов.
- Маршрутизация по сложности: Haiku 4.5 или Sonnet 4.6 для простых ходов, Opus 4.7 для планирования и сложного рассуждения.
- Бета «advisor tool» (Anthropic, март 2026) позволяет соединить executor с более умным advisor посреди генерации.
- Следите за токенайзером Opus 4.7: тот же ценник, что у 4.6, но $\sim 1,0\text{--}1,35\times$ больше токенов на тот же текст. Замеряйте cost-per-task после миграций.
- Batch API для не-real-time нагрузки – скидка 50%.
- Для мульти-агентных сценариев (стиль Anthropic research) ждите $\sim 15\times$ токенов, чем у одиночного чат-агента. Гоняйте мульти-агента, только если ценность ответа эту планку покрывает.

Ресурсы:

- *Open Models have crossed a threshold* (LangChain). GLM-5 и MiniMax M2.7 теперь матчат закрытые frontier-модели на ключевых agent-задачах (file ops, tool use, instruction following). Перед фиксацией модели и стратегии маршрутизации – обязательно.

На что фокусироваться: границы prompt caching, правила маршрутизации моделей, batch vs real-time, жесткий бюджет cost-per-task.

2. Latency

- Параллельные tool calls. В системном промпте Anthropic research дословно: «you MUST use parallel tool calls when creating multiple sub-agents». То же касается ваших агентов.
- Streaming частичных выходов в UI через `stream_mode="updates"` LangGraph.
- Sub-agent fan-out – крупнейший рычаг latency: 60-шаговый последовательный агент превращается в 10 шагов lead + 5 параллельных по 10 шагов.

На что фокусироваться: где параллелизм безопасен, где streaming меняет UX, как fan-out взаимодействует со стоимостью.

3. Безопасность и песочницы

- Все code execution – в песочнице: Modal, E2B, Daytona, LangSmith Sandboxes (private preview, март 2026). Никогда не делайте `exec()` выхода модели в основном процессе.
- Credentials брокерятся вне контекста модели (паттерн Managed Agents, Composio для SaaS-авторизаций).

- Hooks как guardrails: `PreToolUse`-хуки, блокирующие деструктивный Bash, regex-блок секретов, валидация путей записи.
- HITL-interrupts на любое необратимое действие (`LangGraph interrupt()` + `HumanInTheLoopMiddleware`, Claude Agent SDK permission prompts).

Ресурсы:

- *Modal docs* – дефолтная песочница для Python.
- *E2B docs* – code-execution песочницы под AI-агентов.
- *Beyond permission prompts: making Claude Code more secure and autonomous* (Anthropic), 20 октября 2025. Фундаментальный пост про sandboxing.
- *Claude Code auto mode: a safer way to skip permissions* (Anthropic), 25 марта 2026. Что меняется при unattended-режиме.

На что фокусироваться: какие действия обратимы, какие требуют человеческой апрувки, и как сделать так, чтобы модель никогда не видела credential, который использует.

4. Мониторинг и drift

- 100% trace sampling на низких объемах; 1–10% stratified sampling на ошибках при высоких.
- Алерты на: cost per request, tool-call failure rate, среднюю оценку LLM-judge (ежедневно), p95 latency, регрессии оценок.
- Re-baseline оценок после каждого апгрейда модели.
- Anthropic engineering blog предупреждает: «harnesses encode assumptions about what Claude can't do on its own; those assumptions go stale as models improve» (пример с «context anxiety» при Sonnet 4.5 → Opus 4.5).

На что фокусироваться: на что алертить vs что просто логировать, как ловить инвалидацию prompt-cache, как замечать harness ossification, когда модель уже ушла вперед.

5. Resilience

- Durable execution (Inngest, Temporal или LangGraph PostgresSaver) необсуждаем для любого агента, который бежит >60 секунд.
- Чекпоинт после каждого узла. Rewind и fork должны быть возможны. Pydantic Deep Agents и LangGraph поддерживают это. Session log Claude Agent SDK – эквивалент.

Ресурсы:

- *How My Agents Self-Heal in Production* (LangChain) – работающий пайплайн, который ловит регрессии после каждого деплоя, диагностирует причину и открывает фикс-PR без участия человека до ревью.

На что фокусироваться: какие сбои восстанавливаются автоматически, какие требуют эскалации, как тестировать путь resume до того, как продакшн его запросит.

Чекпоинт Фазы 5

Эта фаза не кончается. Но к моменту, когда вы можете идти в продакшн, у вас должно быть:

- Prompt caching на системном промпте, CLAUDE.md и tools.
- Слой маршрутизации моделей с жесткими бюджетами cost-per-task и алертингом.
- Песочница для всех code-execution и брокер credentials, держащий секреты вне контекста.
- Хуки, блокирующие деструктивные действия и форсирующие апрув на необратимых.
- Trace sampling, drift-алерты, ритуал re-baselining на каждом апгрейде.
- Слой durable execution, при котором kill процесса – non-event.

Рекомендации

Если учить только один фреймворк

LangGraph 1.0 + Deep Agents. Самый общий, runtime самый зрелый сегодня (PostgresSaver, time-travel, durable execution, OTEL-friendly observability через LangSmith), model-agnostic. Абстракция (state graph + middleware) переносится. Точка.

Если изучать одну обвязку как референс

Claude Agent SDK + Claude Code. Это эталон. CLAUDE.md, Skills, sub-agents, hooks, plan mode, filesystem-as-memory. Любая другая обвязка в 2026 сходится к этим примитивам. Используйте Claude Code ежедневно, читайте его доки, изучайте open-source компендиумы.

Если читать одно по контексту

Anthropic «Effective context engineering for AI agents» (сент 2025).

Если два – добавьте «**Context Engineering for Agents**» от LangChain ради рамки Write/Select/Compress/Isolate.

Если выбрать один observability-инструмент

- **LangSmith** – если остаетесь в LangGraph.
- **Braintrust** – если хотите framework-agnostic CI gating.
- **Inspect** – если нужна benchmark-grade строгость (а она вам в итоге нужна).

Что пропустить в 2026

- AutoGen v0.4 (мерджнут в Microsoft Agent Framework, community-линия – AG2; ни то, ни другое сильным дефолтом не является).
- OpenAI Swarm (официально заменен, явно «not production-ready» в README).
- Assistants API (sunset в середине 2026).
- Свой vector store или память – до того, как замеры реальную проблему recall.
- «No-code» agent-платформы – кроме одноразовых поделок.

Использовать только под конкретную причину

- **CrewAI.** Самый быстрый путь от идеи к прототипу, хрупок в продакшне. Хакатоны и демо.
- **OpenAI Agents SDK.** ОК, если вы заперты в OpenAI. Апрель 2026 добавил sandbox и harness, но вы по-прежнему привязаны к моделям OpenAI.
- **Pydantic AI / Pydantic Deep Agents.** Берите, если вы строгая FastAPI-команда с типами.
- **Mastra.** Если команда – TypeScript и не может Python. v1.0 январь 2026, YC W25, 22k+ stars, делает команда Gatsby.
- **Smolagents.** Лучший учебный инструмент для code-agent паттернов (1000-строчная кодбаза легко хакается). Слаб в проде.

- **DSPy 3.0 + GEPA.** Когда есть метрика и нужно программно оптимизировать промпты и топологию. GEPA обходит RL на 6% при 35× меньше rollouts (oral на ICLR 2026).
- **Letta / MemGPT.** Если нужна OS-style self-managed память между сессиями. Иначе filesystem + Mem0 проще.

Бенчмарки на закладку (цифры май 2026)

- **SWE-bench Verified:** Claude Opus 4.7 ≈ 87,6%, GPT-5.5 ≈ 88,7%, Gemini 3.1 Pro ≈ 78,8%.
- **Terminal-Bench 2.0:** GPT-5.5 – 82,7%, Opus 4.7 – ~70%, Gemini 3.1 Pro – ~68%.
- **tau-bench:** Claude Mythos Preview – 89,2%.
- **BrowseComp:** GPT-5.5 – 90,1%, Gemini 3.1 Pro – 85,9%, Opus 4.7 – 79,3% (регресс с 83,7% у 4.6; web research отдавайте GPT-5.5).
- **GAIA / Princeton HAL:** Sonnet 4.5 лидирует с 74,6%.

Тайминг для технически крепкого инженера, нового в агентах

- Неделя 2: Фаза 0. Можете объяснить обязанку обычными словами.
- Неделя 5: Фаза 1. Агент на Claude Agent SDK с одним Skill, одним хуком, одним суб-агентом.
- Неделя 9: Фаза 2. LangGraph deep-agent research analyst с `PostgresSaver` и LangSmith-трейсами.
- Неделя 13: Фаза 3. 1500-строчная мини-обязка, написанная и задокументированная.
- Неделя 17: Фаза 4. Golden датасеты, CI gates, прогон по опубликованному бенчмарку через Inspect.
- Бесконечно: Фаза 5.

При работе на 10–15 часов в неделю умножайте примерно на 2,5×.

Бенчмарки, меняющие план: если не получается повернуть Фазу 1 за 3 недели – проблема в дизайне инструментов (перечитайте «Writing tools for agents»). Если Фаза 2 заняла больше 5 недель – вы пытаетесь параллельно собирать обязанку. Откатитесь на Deep Agents и не сопротивляйтесь.

Подводные камни

Бенчмарки – движущиеся цели и частично игровые

SWE-bench Verified прошел с 1,96% до 80%+ за два года. `pass^k` consistency у tau-bench добавили именно потому, что одиночная точность перестала быть информативной. Любое заявление «X модель показала Y%» воспринимайте совместно с обвязкой, скаффолдом, retry-бюджетом и системным промптом. Не моделью одной.

Мульти-агент переоценен для большинства задач

90,2% Anthropic – это конкретно про breadth-first research. На coding и плотно связанных задачах мульти-агент часто хуже одиночного и сжигает 15× токенов. Дефолт – single-agent + sub-agents под scoped exploration. К полноценному multi-agent – только если задача декомпозируется естественно.

Контр-пример: Anthropic's *Building a C compiler with a team of parallel Claudes* (5 февраля 2026) – коддинг-задача, где параллельные суб-агенты окупались. Multi-agent для кода не мертв, ему нужна правильная декомпозиция.

Флажки спекуляции в источниках 2026

Несколько прогнозов «AI 2027» (OpenBrain \$45B revenue и т.д.) – открыто фикшен, но цитируются как факты. Игнорируйте. Заметки «по горячим следам» после релизов – анекдоты. Сигнал о настроении разработчиков, не бенчмарк.

Ландшафт фреймворков может снова сдвинуться

LangChain дважды менял рамку за 18 месяцев (chains → graphs → harnesses-on-graphs). Pydantic AI, Mastra или Deep Agents за 12 месяцев могут стать сильно крупнее. Ставьте на абстракции (loop, tools, context, sub-agents, durability, traces), а не на конкретную библиотеку. Они переносятся.

MCP в продакшне еще шероховат

Streamable HTTP за load balancers, multi-tenant auth, rate limiting, audit logging – явно в roadmap MCP на 2026, то есть не решено. Закладывайте на следующее поколение transport SEPs ближе к концу 2026 и не вешайте архитектуру на текущую модель сессий.

Поведение моделей меняется между точечными релизами

Строже инструкции и новый токенайзер у Orus 4.7 значат, что промпты от 4.6 могут вести себя по-другому и стоить до 35% дороже на тот же текст. Реплейте трафик на каждом апгрейде.

Eval-сюта будет гнить

Golden-датасет, собранный сегодня, насытится за месяцы. Планируйте рост на 10–20% в квартал из продакшн-падений, не из синтетики. Калибровку человеком над LLM-judge держите бесконечно.

Часть источников – vendor marketing

Опирайтесь на первоисточники (Anthropic engineering blog, LangChain blog, OpenAI announcements, arXiv). Рейтинги «лучшее в 2026» (Alice Labs, Channel.tel, GuruSup, Morph, Vstorm) полезны для триангуляции, но у каждого – коммерческий интерес. Где они сходятся между собой и с инженерными первоисточниками, консенсусу можно доверять.

Заключение

Что вас ждет после этого роадмапа?

Без сахара: за 17 недель он не сделает вас principal AI engineer. Но сделает человеком, который умеет собирать и выпускать агентные системы, переживающие продакшн-трафик. А это и есть то, за что компании платят прямо сейчас.

Спрос на инженеров, которые могут отгружать прод-агентов, не падает. По LangChain State of Agent Engineering – 57% команд уже в продакшне, у 89% из них есть observability. Качество – барьер №1 (32%). То есть поле сейчас бутылочно-горлово упирается в инженеров, умеющих делать оценки и обвязки, а не в тех, кто умеет вызывать LLM API.

Цифра Anthropic ловит реальную возможность: та же модель, другая обвязка, 78% vs 42% на CORE.

Этот разрыв – ваша работа.

Сдвиг к harness engineering – крупнейший mispricing в найме сейчас. Компании все еще постят вакансии «prompt engineer», но нужны им инженеры, которые умеют взять frontier-модель и превратить ее в измеримую и долговечную продакшн-систему.

Что я хочу, чтобы вы вынесли

Берите по одному проекту из каждой фазы и **делайте**. Не читайте. Делайте, ломайте, чините, деплойте, кладите LangSmith-трейс и счет с бенчмарка в README. Нанимают тех, кто может показать трейс, а не тех, кто помнит таблицу сравнений фреймворков.

Делитесь тем, что узнали. Опубликуйте post-mortem мини-обвязки. Опубликуйте находки из golden-датасета. Постите цифры с конфигурацией обвязки, их давшей. Обучение – самый быстрый способ учиться, и оно одновременно строит репутацию. Лучшие возможности приходят к видимым инженерам, а не к тем, кто отправил резюме на 500 вакансий.

И не ждите ощущения готовности. Оно не придет. Разрыв между «я читаю LangChain blog» и «я отгрузил deep-agent с PostgresSaver durability» – это место, где большинство инженеров застревает навсегда.

Начинайте подаваться, начинайте строить публично, начинайте отгружать в момент, когда у вас есть рабочий агент. Даже маленький. Рынок не вознаграждает совершенство. Он вознаграждает инженеров, которые могут заставить модель сделать что-то реальное и доказать, что регресса не было.

17 недель достаточно, чтобы все изменилось, если вкладывать работу.

Просто продолжайте строить и продолжайте мерить то, что строите.

Примечание автора оригинала: статья написана автором по личным заметкам, накопленным за 2–3 месяца, и отредактирована Minimax 2.7. У автора есть пайплайн контента в Obsidian, который превращает рукописные и набранные заметки в публикации в его стиле.